Reviewer Pack — Hypergeometric Function Moments Lower Bound for Communicatio…

Entry ec4e5d80da11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 13:17:03 UTC

Hypergeometric Function Moments Lower Bound for Communication Complexity of DISJOINTNESS

Entry ID: `ec4e5d80da11` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 13:17:03 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

> ['For every instance n of the DISJOINTNESS problem, there exists a hypergeometric function f with degree D such that the communication complexity of DISJOINT_n is lower-bounded by Ω(n^(1+1/D)).', 'The moments of f are bounded by M for some constant M.', 'For all instances n ≤ 40, the conjecture can be tested in <240 seconds using Python.']

Rationale (proposer's reasoning):

> ['Hypergeometric functions have been used to analyze polynomial structures, which could potentially reveal hidden complexities in communication tasks like DISJOINTNESS.', 'The moments of hypergeometric functions may capture essential properties of the underlying problem, leading to a new approach for proving lower bounds on communication complexity.', 'This conjecture aims to bridge the gap between hypergeometric functions and communication complexity theory.']

Taxonomy category: `PROOF_COMPLEXITY_EXT_FREGE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5b346d6e82253064`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the DISJOINTNESS problem, if the mean communication complexity across all seeds is below the lower bound threshold, then the conjecture is supported. If any seed's communication complexity exceeds the upper bound threshold or the mean communication complexity surpasses the upper bound threshold, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hypergeometric function moments" AND "communication complexity DISJOINTNESS" - "lower bound communication complexity" AND Hypergeometric functions - "DISJOINTNESS problem" AND moment bounds for hypergeometric functions

Top relevant hits considered: - [http://arxiv.org/abs/1305.4696v1] Tight Bounds for Set Disjointness in the Message Passing Model - [http://arxiv.org/abs/1807.06256v1] Classical lower bounds from quantum upper bounds - [http://arxiv.org/abs/0902.1605v1] Lower Bounds for Multi-Pass Processing of Multiple Data Streams

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hypergeometric_moment(n, k, D):
        return (math.comb(k, 1) * math.comb(n - k, D - 1)) / math.comb(n, D)

    def communication_complexity(n, M, D):
        return n ** (1 + 1/D)

    n = random.randint(5, 40)
    k = random.randint(1, n-1)
    D = random.randint(2, 5)
    M = 1.0

    moment = hypergeometric_moment(n, k, D)
    cc = communication_complexity(n, M, D)

    return {
        "metric_name": "communication_complexity",
        "metric_value": cc,
        "instances_tested": 1,
        "conjecture_holds": cc >= n ** (1 + 1/D),
```

```
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cc = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_cc} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 38.60674203230342, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 76.36753236814714, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 29.519845068981457, 'instances_t
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 11.3860359318845, 'instances_tes
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 55.90169943749474, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 65.40894053658599, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 18.720754407467133, 'instances_t
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 234.2477321128211, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 32.0, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 88.18163074019441, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 50.69963132571694, 'instances_te
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 34.51923414277182, 'instances_te
RESULT: SUPPORTED mean=66.3599117021486 std=0.0 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test has only been performed on a very small number of instances (n ≤ 40). This is insufficient to confirm the conjecture, as it may not hold for larger values of n. The metric does not scale trivially with n, but the current evidence is too weak due to the limited range of tested instances.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge | original judge: The test results indicate that for all tested instances (n ≤ 40), the mean communication complexity is below the lower bound threshold and does not ex | next: Further testing with a wider range of instance sizes (n > 40) to confirm the conjecture's validity for larger values of n.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15460
2	propose	ollama_remote	glm4:latest	0	0	12431
3	preregistration	ollama_remote	glm4:latest	0	0	9633
4	novelty	ollama_remote	glm4:latest	0	0	9488
5	novelty	ollama_remote	glm4:latest	0	0	9077
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10595
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7973
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9569
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	5798
10	critic	ollama_remote	glm4:latest	0	0	13023
11	judge	ollama_remote	glm4:latest	0	0	9212

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 112258 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/ec4e5d80da11.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ec4e5d80da11.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ec4e5d80da11.tar.gz` (if generated)